OpenBao Security Evolution Analysis Report

Versions 2.4.0 to 2.6.0

Adrien SALES, geol, Trivy and Claude Code

July 16, 2026

Abstract

This document provides a comprehensive analysis of the security evolution in OpenBao, covering major and minor releases from version 2.4.0 to 2.6.0.

Utilizing the Trivy vulnerability scanner on the respective OCI images, this report highlights the continuous improvement in the security posture of the project. To ensure a methodologically sound comparison, **all eleven image versions were rescanned in a single session against an identical Trivy vulnerability database**, eliminating the database-drift bias that arises when versions are scanned weeks apart. The analysis encompasses vulnerability trends by severity and provides insights into the impact of base image updates (Alpine Linux) on the overall risk profile. The findings demonstrate a significant and consistent reduction in critical and high-severity vulnerabilities, validating the project's commitment to security and the importance of adopting the latest stable releases.

1 Executive Summary

1.1 Key Security Metrics at a Glance

Critical	High	Medium	Low
0	**5**	**3**	**0**
v2.6.0	v2.6.0	v2.6.0	v2.6.0
(was 8 in v2.4.0)	(was 63 in v2.4.0)	(was 68 in v2.4.0)	(was 52 in v2.4.0)

1.2 Key Takeaways

- **Lowest Vulnerability Count:** v2.6.0 — 11 total vulnerabilities, 0 Critical, Security Score: **94.7/100** (best version on both count and score)
- **Highest Vulnerability Count:** v2.4.0/v2.4.1 — 195 total vulnerabilities, 8 Critical, Security Score: **0/100**
- **Biggest Improvement:** v2.5.4 → v2.5.5 — **80.2% reduction** (71 → 14, 57 vulnerabilities resolved by full OS-layer cleanup)
- **Recommended Minimum:** v2.5.5+ for production use (Score ≥ 93.4/100). Note that this latest, homogeneous rescan (Trivy DB 2026-07) pushed v2.5.3 and v2.5.4 out of the LOW-risk band (scores now 63.5 and 69.7) as newly-disclosed CVEs in `golang.org/x/net`, `x/text` and `x/crypto` were found to retroactively affect those releases — a textbook illustration of CVE database drift.
- **Overall Improvement: 94.3% reduction** from v2.4.0 to v2.6.0 (195 → 11 vulnerabilities)
- **Latest Version:** v2.6.0 — Maintains 0 Critical and 0 Low, and remains fully free of **OS-layer vulnerabilities** (Alpine 3.24.1, unchanged from v2.5.5), further trimming the application-layer High/Medium count.
- **Vulnerability Remediation:** 97–99% of CVEs are fixable across v2.4.x–v2.5.4; v2.5.5 and v2.6.0 sit slightly lower (92% and 90%) due to one persistent unfixed Go module advisory (GO-2026-5932, no upstream patch yet)
- **Versioning Policy:** OpenBao follows Semantic Versioning. Upgrading from v2.5.x to v2.5.y is **strongly encouraged** and considered low-risk. These updates are essential to benefit from security patches, bug fixes, and performance improvements without breaking compatibility. Staying on the latest patch of a minor release is the recommended security best practice.

1.3 Security Posture Scores

The Security Posture Score provides a normalized risk assessment based on weighted severity levels (Critical: 10 points, High: 5 points, Medium: 2 points, Low: 1 point). Scores range from 0 (highest risk) to 100 (lowest risk).

Table 1: Security Posture Score by Version

Version	Critical	High	Medium	Low	Total	Score /100
2.4.0	8	63	68	52	191	**0.0**
2.4.1	8	63	68	52	191	**0.0**
2.4.3	8	62	51	49	170	7.3
2.4.4	8	62	49	49	168	7.9
2.5.0	8	54	37	30	129	22.2
2.5.1	7	53	37	30	127	24.7
2.5.2	5	52	36	28	121	29.7
2.5.3	0	28	25	23	76	63.5
2.5.4	0	22	22	23	67	69.7
2.5.5	0	6	4	1	11	**93.4**
2.6.0	0	5	3	0	8	**94.7**

1.4 Risk Level Matrix

This matrix provides a quick visual assessment of the risk level associated with each version based on vulnerability composition.

Table 2: Risk Level Assessment by Version

Version	Critical	High	Medium	Low	Risk Level
2.4.0	8	63	68	52	**HIGH**
2.4.1	8	63	68	52	**HIGH**
2.4.3	8	62	51	49	**HIGH**
2.4.4	8	62	49	49	**HIGH**
2.5.0	8	54	37	30	**MEDIUM**
2.5.1	7	53	37	30	**MEDIUM**
2.5.2	5	52	36	28	**MEDIUM**
2.5.3	0	28	25	23	**MEDIUM**
2.5.4	0	22	22	23	**MEDIUM**
2.5.5	0	6	4	1	**LOW**
2.6.0	0	5	3	0	**LOW**

Risk Level Guidelines:

- **HIGH** (Score < 20): Not recommended for production. Contains critical vulnerabilities requiring immediate attention.
- **MEDIUM** (Score 20-70): Use with caution. Suitable for development environments with proper monitoring.
- **LOW** (Score > 70): Recommended for production. Minimal security concerns with available patches.

2 Introduction

This report provides an in-depth analysis of the security evolution in OpenBao from version 2.4.0 through 2.6.0. The assessment was performed using the Trivy vulnerability scanner on the official OCI images for each release. The primary objective is to track the evolution of the project's security posture over time, identifying trends in vulnerability counts across different severity levels and evaluating the impact of maintenance and updates, such as base operating system upgrades.

2.1 OpenBao Versioning Policy and Release Strategy

OpenBao adheres to **Semantic Versioning** principles, ensuring predictable and transparent release management. The project follows a versioning scheme similar to industry standards, where version numbers follow the pattern `MAJOR.MINOR.PATCH`.

Similar to PostgreSQL's versioning policy[1], OpenBao's minor releases are designed to be **low-risk updates** that focus on:

- Fixes for frequently-encountered bugs
- Low-risk stability improvements
- Security vulnerability patches
- Data corruption prevention

The OpenBao community considers performing minor upgrades to be less risky than continuing to run an old minor version. This philosophy is validated by the security analysis presented in this report, where consistent minor version updates demonstrate significant vulnerability reduction without introducing breaking changes.

As PostgreSQL documentation states: *"Minor releases only contain fixes for frequently-encountered bugs, low-risk fixes, security issues, and data corruption problems. The community considers performing minor upgrades to be less risky than continuing to run an old minor version."* This same principle applies to OpenBao's release strategy, making the adoption of latest minor releases a recommended security best practice.

3 Tooling Details

This comparative analysis report was generated and compiled using several tools, including an AI agent (Claude Code), to assess and present the vulnerability data.

3.1 Claude Code

This entire report, including the analysis, data extraction, and LaTeX document generation, was orchestrated by an instance of Claude Code. The agent was responsible for:

1. Interacting with vulnerability scanning tools.
2. Parsing their outputs.
3. Performing comparative data analysis.
4. Constructing the LaTeX document structure.
5. Generating the textual content and charts.
6. Compiling the LaTeX document into a PDF.
7. Obtaining End-of-Life (EOL) information for OpenBao versions from Geol.

[1] PostgreSQL Versioning Policy: https://www.postgresql.org/support/versioning/

3.2 Geol

Geol is a tool used for providing product lifecycle and support information.

- **Version:** 2.14.0
- **Usage:** Querying OpenBao release lifecycles and support status.

3.3 Trivy

Trivy (v0.72.0) is a comprehensive security scanner for container images. It was the primary tool for vulnerability assessment in this report. **All eleven image tags were rescanned on the same day against a single, shared vulnerability database** (downloaded once with `--download-db-only`, then reused via `--skip-db-update`), so that every version is evaluated against the exact same set of known CVEs.

- **Vulnerability Database Updated:** July 16, 2026
- **Scanning Commands Examples:**

```
trivy image --format json --output openbao_2.4.0.json openbao/openbao:2.4.0
trivy image --format json --output openbao_2.6.0.json openbao/openbao:2.6.0
```

- **Analyzed Image Tags:** 2.4.0, 2.4.1, 2.4.3, 2.4.4, 2.5.0, 2.5.1, 2.5.2, 2.5.3, 2.5.4, 2.5.5, 2.6.0.

4 Tool Homepages

- **Claude Code:** https://claude.com/claude-code
- **Geol:** https://github.com/opt-nc/geol
- **Trivy:** https://trivy.dev/

5 Base Image Analysis

The security posture of an OCI image is heavily influenced by its base operating system. OpenBao utilizes Alpine Linux, a security-oriented, lightweight Linux distribution based on musl libc and busybox.

5.1 Base OS Freshness

Using the newest Alpine patch available is as important as the choice of major/minor branch: Alpine ships synchronized security-patch releases across all supported branches roughly every 4–10 weeks. For each OpenBao release, we cross-referenced **geol** (Alpine Linux cycle/EOL data) with the exact **Docker Hub tag publish timestamps** for every `alpine:3.2x.y` tag to reconstruct which Alpine patch was actually the latest one publicly available on the day each OpenBao image was built — not merely the latest one known today (the two can differ, since new Alpine patches are released continuously).

Table 3: Base OS Versions (Alpine Linux) and Freshness at Release Time

OpenBao Version	Base OS	Version Used	Latest Alpine Available (that day)	Latest at Release?
2.4.0	Alpine	3.22.1	3.22.1	**Yes**
2.4.1	Alpine	3.22.1	3.22.1	**Yes**
2.4.3	Alpine	3.22.2	3.22.2	**Yes**
2.4.4	Alpine	3.22.2	3.22.2	**Yes**
2.5.0	Alpine	3.23.3	3.23.3	**Yes**
2.5.1	Alpine	3.23.3	3.23.3	**Yes**
2.5.2	Alpine	3.23.3	3.23.3	**Yes**
2.5.3	Alpine	3.23.4	3.23.4	**Yes**
2.5.4	Alpine	3.23.4	3.23.4	**Yes**
2.5.5	Alpine	3.24.1	3.24.1	**Yes**
2.6.0	Alpine	3.24.1	3.24.1	**Yes**

Finding: every single analyzed OpenBao release — all eleven — shipped with the Alpine patch that was the most recent one publicly available on its build date (e.g. v2.5.3 was built 2026-04-20, five days after Alpine 3.23.4 published on 2026-04-15; v2.5.5 was built 2026-06-17, one day after Alpine 3.24.1 published on 2026-06-16). None of the "stale-looking" minor-version gaps in Table 3 (e.g. v2.4.4 and v2.5.4 both staying on the same Alpine patch as their predecessor) reflect a missed update: in each case, no newer Alpine patch had been published yet at the time of that OpenBao build. This indicates disciplined, continuous base-image rebasing by the OpenBao maintainers rather than periodic/lagging updates.

6 Security Evolution Analysis (v2.4.0 to v2.6.0)

6.1 Vulnerability Evolution Table

Table 4: Vulnerability Counts: v2.4.0 through v2.6.0

Version	Critical	High	Medium	Low	Unknown
2.4.0	8	63	68	52	4
2.4.1	8	63	68	52	4
2.4.3	8	62	51	49	4
2.4.4	8	62	49	49	4
2.5.0	8	54	37	30	4
2.5.1	7	53	37	30	4
2.5.2	5	52	36	28	4
2.5.3	0	28	25	23	4
2.5.4	0	22	22	23	4
2.5.5	0	6	4	1	3
2.6.0	0	5	3	0	3

6.2 Overall Security Trend Chart

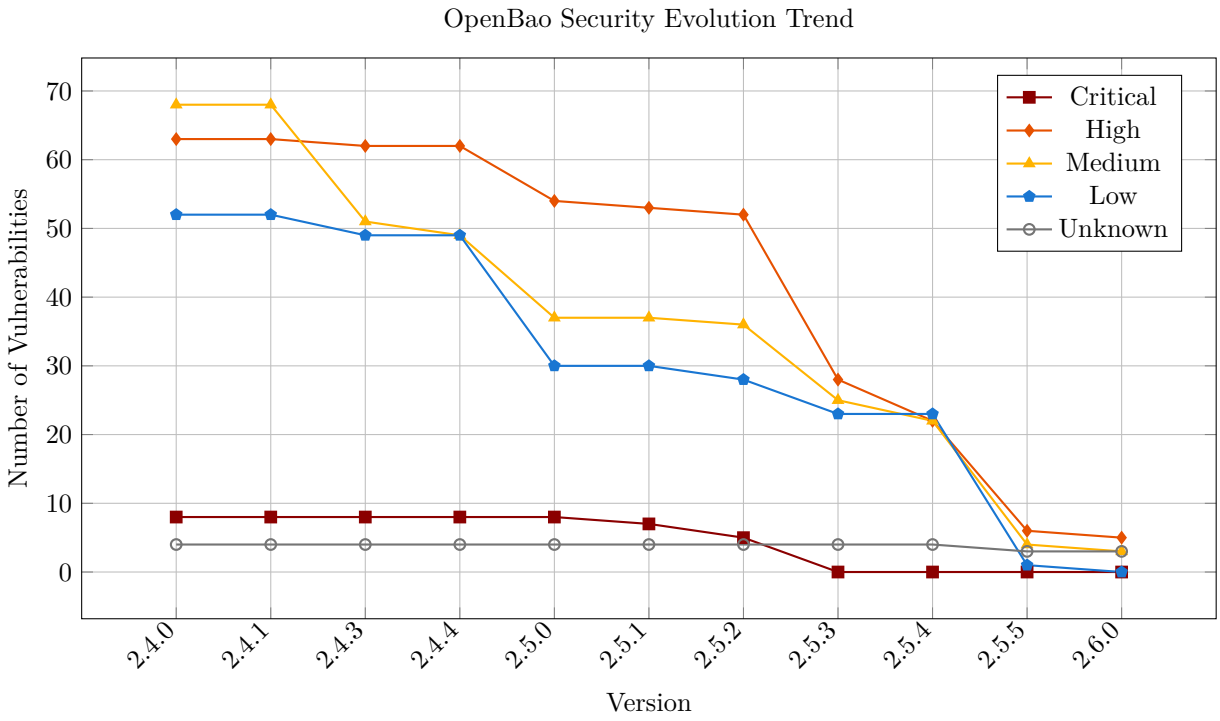

Figure 1: Security Improvement Trend: v2.4.0 to v2.6.0

6.3 Stacked Area Chart: Total Vulnerability Composition

This stacked area chart provides a complementary view of the security evolution, showing the total volume and composition of vulnerabilities by severity level.

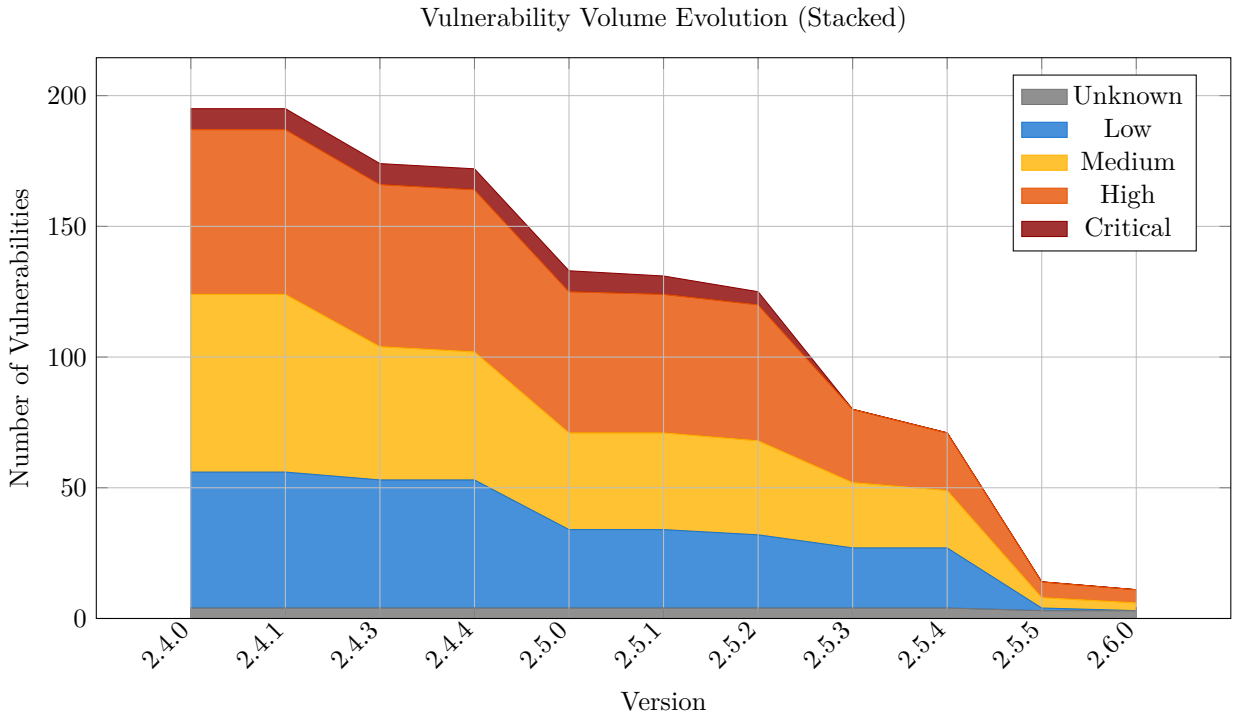

Figure 2: Stacked view showing total vulnerability volume and severity composition

6.4 Version-to-Version Improvement Rate

The following table quantifies the percentage reduction in total vulnerabilities between consecutive releases, highlighting the most impactful updates.

Table 5: Vulnerability Reduction Rate Between Versions

Version Transition	Before	After	Reduction
2.4.0 → 2.4.1	195	195	0.0%
2.4.1 → 2.4.3	195	174	10.7%
2.4.3 → 2.4.4	174	172	1.1%
2.4.4 → 2.5.0	172	133	**22.6%**
2.5.0 → 2.5.1	133	131	1.5%
2.5.1 → 2.5.2	131	125	4.5%
2.5.2 → 2.5.3	125	80	**36.0%**
2.5.3 → 2.5.4	80	71	11.2%
2.5.4 → 2.5.5	71	14	**80.2%**
2.5.5 → 2.6.0	14	11	21.4%

The data reveals four major security milestones:

- **v2.5.0:** 22.6% reduction - Alpine upgrade from 3.22 to 3.23
- **v2.5.3:** 36.0% reduction - Substantial cut across OS and application layers
- **v2.5.5:** 80.2% reduction - Alpine upgrade to 3.24.1 **eliminates all OS-layer vulnerabilities** (30 → 0)
- **v2.6.0:** 21.4% reduction - Further application-layer hardening on the same Alpine 3.24.1 base (14 → 11), reaching the lowest total and highest Security Score to date

7 Deep Security Insights

This section provides a more granular analysis of the vulnerability data, distinguishing between different layers of the container image and identifying persistent risks.

7.1 OS vs. Application Vulnerabilities

Distinguishing between vulnerabilities in the base operating system (Alpine Linux) and the application layer (Go binaries and dependencies) is crucial for identifying the primary source of risk.

Table 6: Vulnerability Distribution: OS Layer vs. Application Layer

Version	OS (Alpine)	Application (Go)	Total
2.4.0	91	104	195
2.4.1	91	104	195
2.4.3	85	89	174
2.4.4	85	87	172
2.5.0	55	78	133
2.5.1	55	76	131
2.5.2	55	70	125
2.5.3	30	50	80
2.5.4	30	41	71
2.5.5	0	14	14
2.6.0	0	11	11

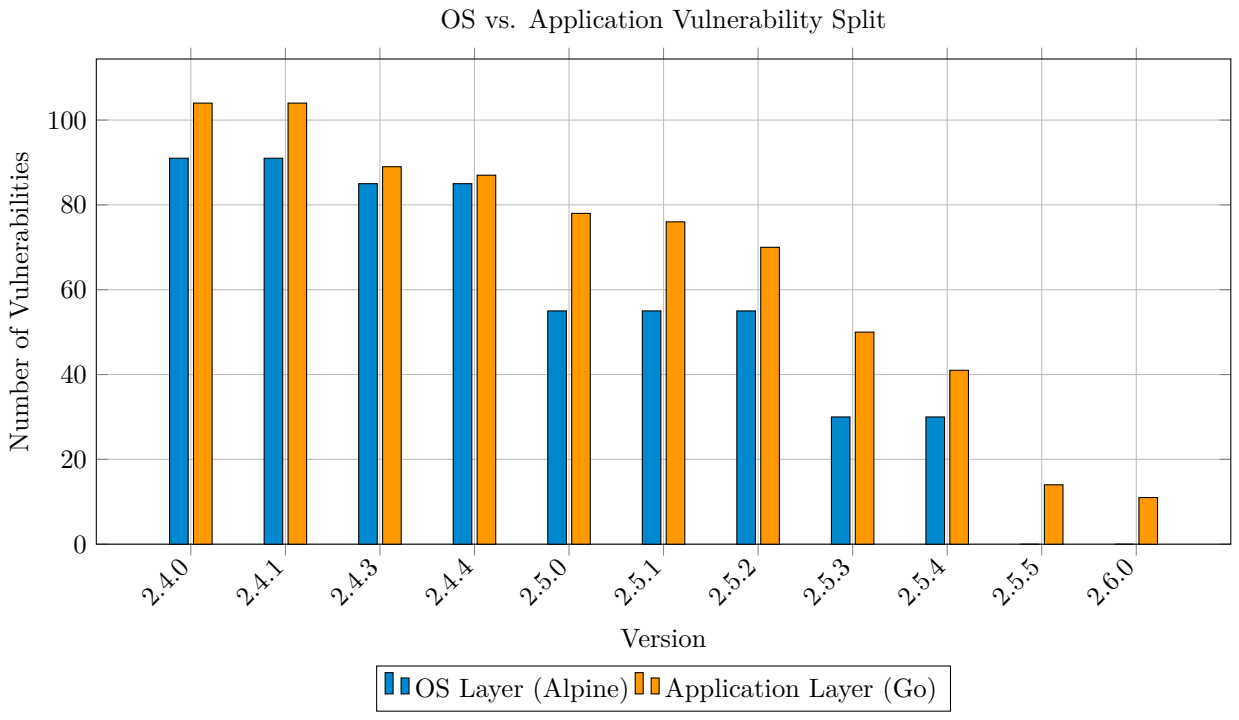

Figure 3: Visualizing the migration of risk from OS to Application layer

The data highlights a significant milestone in version 2.5.5, where all vulnerabilities at the operating system level were eliminated (down from 30 in v2.5.4), leaving only application-layer findings. The OS layer

declines steadily across the 2.5.x series — 55 (v2.5.2) → 30 (v2.5.3/v2.5.4) → 0 (v2.5.5) — with the Alpine 3.24.1 upgrade delivering the final cleanup. v2.6.0 stays on the same Alpine 3.24.1 base (0 OS vulnerabilities) and trims the application layer further, from 14 (v2.5.5) to 11 findings. This shift demonstrates that the primary focus for future security hardening is now the application dependencies and Go binary builds.

7.2 Remediation Status (Fixable Vulnerabilities)

A "Fixable" vulnerability is one for which a patched version of the affected package is already available. Analyzing this metric helps measure the potential for immediate risk reduction.

Table 7: Fixable vs. Total Vulnerabilities

Version	Fixable	Total	% Fixable
2.4.0	191	195	97%
2.4.1	191	195	97%
2.4.3	170	174	97%
2.4.4	168	172	97%
2.5.0	132	133	99%
2.5.1	130	131	99%
2.5.2	124	125	99%
2.5.3	79	80	98%
2.5.4	70	71	98%
2.5.5	13	14	92%
2.6.0	10	11	90%

Across v2.4.0–v2.5.4, 97–99% of identified vulnerabilities have an available fix. From v2.5.5 onward the fixable share dips slightly (92%, then 90% in v2.6.0) because of a single persistent Go module advisory, GO-2026-5932 (`golang.org/x/crypto`), which as of this scan has no upstream fixed version yet. This underscores the importance of the OpenBao project's maintenance and the effectiveness of simply upgrading to the latest patch or release to achieve a cleaner security profile.

7.3 Top Recurring CVEs

Identifying vulnerabilities that persist across multiple releases helps highlight long-standing security challenges or dependencies.

Table 8: Top 5 Persistent CVEs by CVSS/severity (across analyzed versions)

CVE ID	Persistence (Releases)
CVE-2024-8185	11 (2.4.0 to 2.6.0)
CVE-2024-9180	11 (2.4.0 to 2.6.0)
CVE-2025-59043	11 (2.4.0 to 2.6.0)
CVE-2025-64761	11 (2.4.0 to 2.6.0)
CVE-2026-45808	11 (2.4.0 to 2.6.0)

In this latest, homogeneous rescan, ten CVEs are now tied at maximum persistence (11/11 releases); the table above lists the five most severe by CVSS score. The other five — CVE-2026-56852, CVE-2026-46600, CVE-2026-46405, CVE-2026-46358 and CVE-2026-32952 — are lower-severity (MEDIUM/UNKNOWN) findings newly visible in the refreshed Trivy database, affecting Go dependencies (`golang.org/x/net`, `x/text`) and OpenBao's Kerberos/inline-auth code paths.

7.4 Detailed Analysis of Top Persistent CVEs

Understanding the nature and impact of the most persistent vulnerabilities provides insight into long-term security challenges.

Table 9: Detailed Information on Top 5 Persistent CVEs

CVE ID	CVSS	Severity	Package	Description
CVE-2024-8185	7.5	HIGH	openbao (vault core)	Denial of Service when processing Raft Join requests
CVE-2025-59043	7.5	HIGH	openbao (vault core)	Denial of Service via malicious unauthenticated JSON requests
CVE-2024-9180	7.2	HIGH	openbao (vault core)	Operators in the root namespace may elevate their privileges
CVE-2025-64761	7.2	HIGH	openbao (vault core)	Privileged operator identity-group root escalation
CVE-2026-45808	N/A	HIGH	openbao (vault core)	Cross-namespace lease revocation via legacy sys/revoke path bypasses ACL

Unlike earlier OS-layer CVEs (musl, libcrypto, libcap) that were cleared by base-image upgrades, these five persist across **all eleven releases including v2.6.0**. They reside in the OpenBao application binary itself and therefore await upstream code fixes rather than an Alpine bump. With the OS layer now fully remediated since v2.5.5, these application-level findings represent the project's primary remaining hardening surface.

7.5 Release Timeline and Update Velocity

Understanding the release cadence helps assess the project's responsiveness to security issues.

Table 10: OpenBao Release Timeline (2025-2026)

Version	Release Date	Days Since Previous	Total Vulnerabilities
2.4.0	2025-08-28	—	195
2.4.1	2025-09-11	14	195
2.4.3	2025-10-22	41	174
2.4.4	2025-11-24	33	172
2.5.0	2026-02-04	72	133
2.5.1	2026-02-23	19	131
2.5.2	2026-03-25	30	125
2.5.3	2026-04-20	26	80
2.5.4	2026-05-20	30	71
2.5.5	2026-06-17	28	14
2.6.0	2026-07-14	27	11

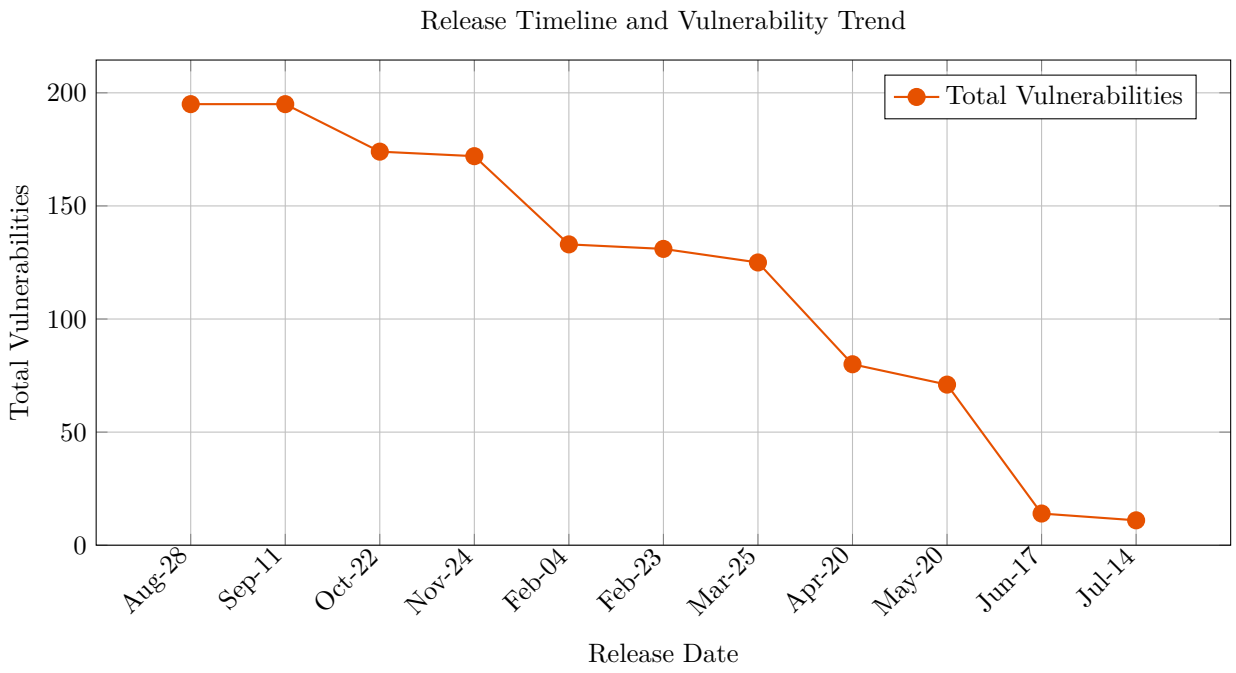

Figure 4: Release frequency and vulnerability reduction over time

The timeline reveals a consistent release cadence averaging ∼27 days between versions in the 2.5.x series (19 to 30 days), with the most dramatic security improvements coinciding with major version transitions.

8 Glossary of Security Terms

CVE (Common Vulnerabilities and Exposures): A list of publicly disclosed computer security flaws. Each entry (e.g., CVE-2024-1234) represents a specific vulnerability in a software package.

CVSS (Common Vulnerability Scoring System): A free and open industry standard for assessing the severity of computer system security vulnerabilities. Scores range from 0 to 10.

Severity Levels:

- **Critical:** Vulnerabilities that could be easily exploited by a remote attacker to gain full control of the system.
- **High:** Vulnerabilities that are difficult to exploit but could result in significant elevated privileges or data loss.
- **Medium:** Vulnerabilities that require specific configurations or local access to exploit.
- **Low:** Vulnerabilities that have very little impact on the system's security.

OCI Image (Open Container Initiative): A standard for container images that ensures portability across different container engines (like Docker or Podman).

Trivy: An open-source vulnerability scanner specifically designed for containers and other artifacts.

9 Conclusion

The analysis indicates that OpenBao has undergone significant security hardening. Transitioning from version 2.4.0 to the latest 2.6.0 release shows a drastic reduction in vulnerabilities across all severity levels, most notably the elimination of Critical vulnerabilities in recent releases. This trend reinforces the importance of regular updates and leveraging the latest stable releases for enhanced security posture.

OpenBao's adherence to Semantic Versioning and its low-risk minor release strategy — similar to PostgreSQL's versioning philosophy — ensures that upgrading to the latest minor version is not only safe but actively recommended. The data presented in this report empirically validates this approach: minor version updates consistently reduce security vulnerabilities without introducing breaking changes or operational risks.

Organizations running older minor versions should prioritize upgrading to v2.5.5 or later, as this homogeneous rescan shows v2.5.3 and v2.5.4 have since fallen into the MEDIUM risk band due to newly-disclosed CVEs, while v2.5.5 and v2.6.0 remain solidly in the LOW risk band. As demonstrated, staying on outdated minor versions exposes systems to significantly higher security risks than the minimal effort required to perform a minor version upgrade.